Task:19 Web and System Performance Tuning

Objective

Learn about different methods used to improve **webpage and Linux system performance**, including:

1. PageSpeed
2. Gzip compression
3. Tuned daemon
4. Performance testing and verification

Requirements

- AWS EC2 instance running RHEL
- Apache (`httpd`) web server
- A simple `index.html` webpage
- AWS Security Group allowing HTTP traffic on port 80
- Publicly accessible webpage URL
- PageSpeed testing tool
- Gzip compression testing tool
- Tuned daemon

Procedure

Step 1 – Launch the EC2 Instance

An AWS EC2 instance running RHEL was launched to perform the web and system performance tuning experiments.

The instance was accessed through SSH.

Step 2 – Install and Start Apache

The Apache web server was installed on the RHEL EC2 instance using:

```
sudo dnf install httpd -y
```

The Apache service was enabled and started:

```
sudo systemctl enable --now httpd
```

The service status was verified using:

```
sudo systemctl status httpd
```

The Apache service was confirmed to be **active and running**.

Step 3 – Create the Test Webpage

A simple webpage was created in Apache's default document root:

```
/var/www/html
```

The webpage was created using:

```
sudo vi /var/www/html/index.html
```

The webpage was successfully accessed through the EC2 public IP address, confirming that Apache was serving the website correctly.

Step 4 – Perform the Initial PageSpeed Test

The publicly accessible webpage URL was submitted to the PageSpeed testing tool to obtain a **baseline performance result** before applying additional tuning methods.

The following results were recorded:

Metric	Result
Performance	**100**
Accessibility	**73**
Best Practices	**78**
SEO	**90**
First Contentful Paint	**0.6 s**
Largest Contentful Paint	**0.6 s**
Total Blocking Time	**0 ms**
Cumulative Layout Shift	**0**
Speed Index	**1.1 s**

The baseline results were recorded for comparison.

Report from Sep 25, 2026, 10:16:07 AM

http://13.203.199.137/ Analyze

Mobile Desktop

Discover what your real users are experiencing No Data

http://13.203.199.137 Test webpage

Results for: *http://13.203.199.137/*

Gzip Is Enabled

Original Size:	321 B
Gzip Size (compressed):	222 B
Compression %:	**30.84% is compressed**
HTTP Status:	200
Request Time:	0.4062 ms
Compression Time:	0.0001 s
Content Type:	text/html
Server:	Apache/2.4.63 (Red Hat Enterprise Linux)

Gzip Compression

Step 5 – Check Apache Gzip Support

Apache's `mod_deflate` module was checked using:

```
httpd -M | grep deflate
```

The following output was obtained:

```
deflate_module (shared)
```

This confirmed that Apache's compression module was available.

Step 6 – Check the Initial Compression Status

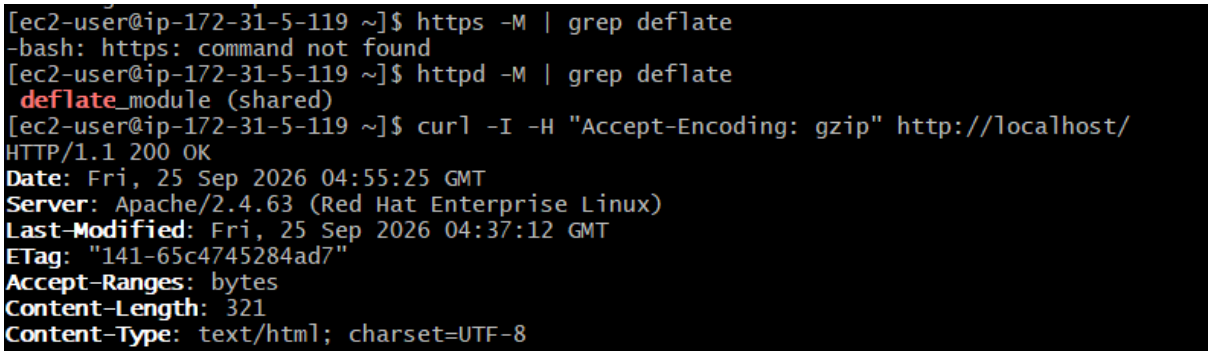
Before enabling Gzip, the HTTP response was checked using:

```
curl -I -H "Accept-Encoding: gzip" http://localhost/
```

The response did not contain:

```
Content-Encoding: gzip
```

This confirmed that Gzip compression was not being applied.



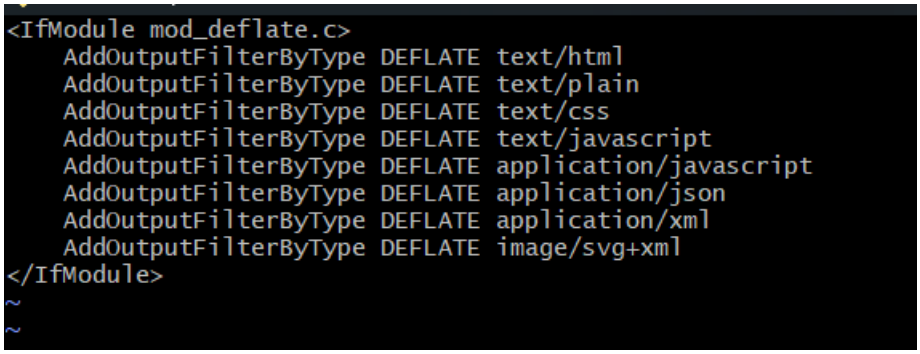
Step 7 – Configure Gzip Compression

A new Apache configuration file was created:

```
sudo vi /etc/httpd/conf.d/gzip.conf
```

The following configuration was added:

```
<IfModule mod_deflate.c>
    AddOutputFilterByType DEFLATE text/html
    AddOutputFilterByType DEFLATE text/plain
    AddOutputFilterByType DEFLATE text/css
    AddOutputFilterByType DEFLATE text/javascript
    AddOutputFilterByType DEFLATE application/javascript
    AddOutputFilterByType DEFLATE application/json
    AddOutputFilterByType DEFLATE application/xml
    AddOutputFilterByType DEFLATE image/svg+xml
</IfModule>
```



Step 8 – Test and Apply Gzip Configuration

The Apache configuration was checked using:

```
sudo apachectl configtest
```

The result was:

```
Syntax OK
```

Apache was then restarted:

```
sudo systemctl restart httpd
```

Step 9 – Verify Gzip Compression

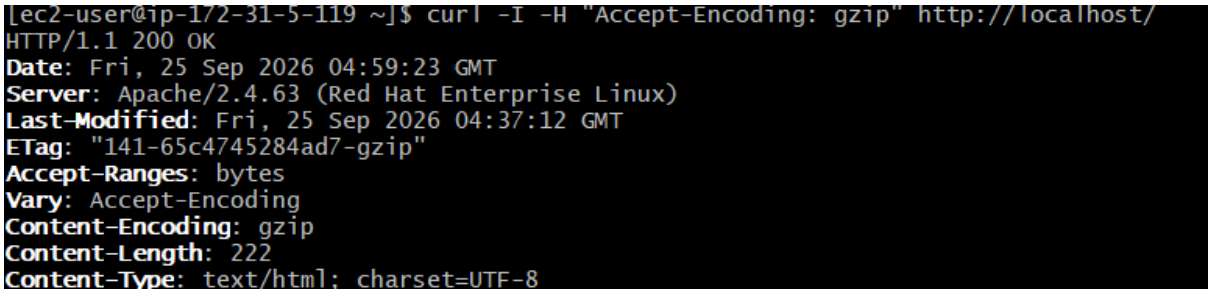
The HTTP response was tested again:

```
curl -I -H "Accept-Encoding: gzip" http://localhost/
```

The response now contained:

```
Content-Encoding: gzip
```

This confirmed that Gzip compression was successfully enabled.



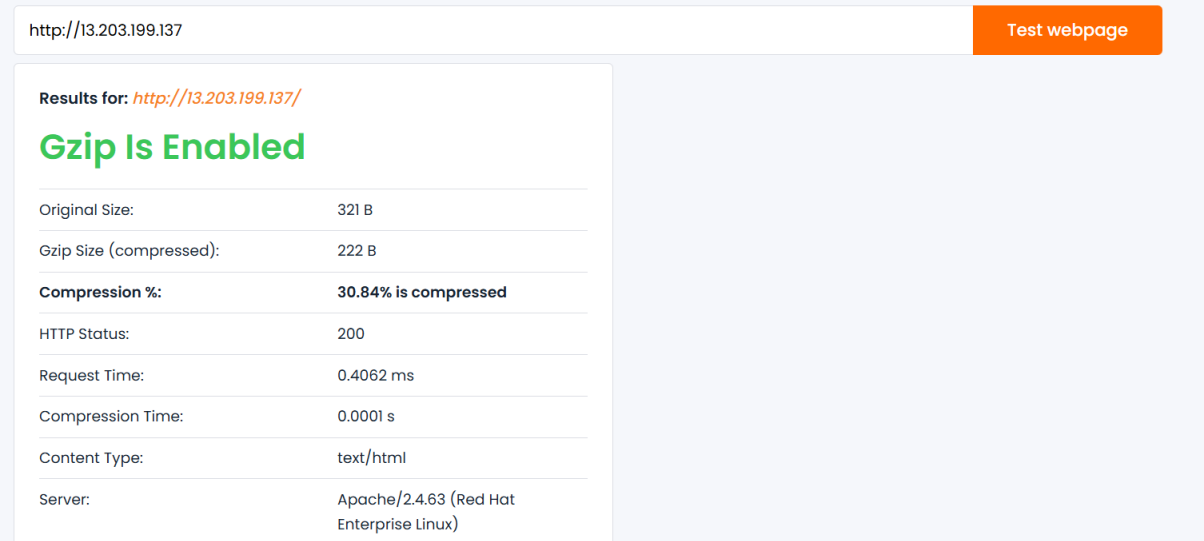
Step 10 – Test Gzip Using a Compression Testing Tool

The webpage was tested using a Gzip compression testing tool.

The results were:

Test	Result
Gzip	**Enabled**
Original size	**321 B**
Compressed size	**222 B**
Compression	**30.84%**
HTTP Status	**200**
Content Type	**text/html**

The test confirmed that the webpage response was successfully compressed.



Tuned Daemon

Step 11 – Check Tuned Installation

The Tuned package was already installed on the RHEL system.

The Tuned service was checked and confirmed to be **active and running**.

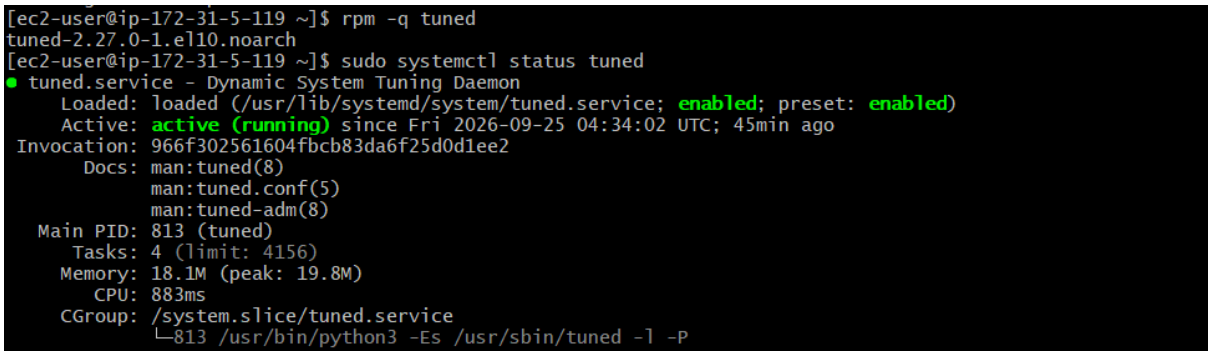
Step 12 – Check the Current Tuned Profile

The current active profile was checked using:

```
tuned-adm active
```

Initially, the following profile was active:

```
Current active profile: virtual-guest
```



Step 13 – Check Available Tuned Profiles

The available Tuned profiles were displayed using:

```
tuned-adm list
```

The `aws` profile was available with the description:

```
aws - Optimize for aws ec2 instances
```

Since the server was running on AWS EC2, this profile was selected for testing.

```
[ec2-user@ip-172-31-5-119 ~]$ tuned-adm active
Current active profile: virtual-guest
[ec2-user@ip-172-31-5-119 ~]$ tuned-adm list
Available profiles:
- accelerator-performance     - Throughput performance based tuning with disabled higher latency STOP states
- aws                         - Optimize for aws ec2 instances
- balanced                    - General non-specialized tuned profile
- balanced-battery            - Balanced profile biased towards power savings changes for battery
- desktop                     - Optimize for the desktop use-case
- hpc-compute                 - Optimize for HPC compute workloads
- intel-sst                   - Configure for Intel Speed Select Base Frequency
- latency-performance         - Optimize for deterministic performance at the cost of increased power consumption
- network-latency             - Optimize for deterministic performance at the cost of increased power consumption, focu
- network-throughput          - Optimize for streaming network throughput, generally only necessary on older CPUs or 4
- optimize-serial-console     - Optimize for serial console use.
- powersave                   - Optimize for low power consumption
- throughput-performance      - Broadly applicable tuning that provides excellent performance across a variety of comm
- virtual-guest               - Optimize for running inside a virtual guest
- virtual-host                - Optimize for running KVM guests
Current active profile: virtual-guest
```

Step 14 – Apply the AWS Tuned Profile

The AWS profile was applied using:

```
sudo tuned-adm profile aws
```

The active profile was then verified:

```
tuned-adm active
```

The result was:

```
Current active profile: aws
```

This confirmed that the AWS Tuned profile was successfully applied.

```
[ec2-user@ip-172-31-5-119 ~]$ sudo tuned-adm profile aws
[ec2-user@ip-172-31-5-119 ~]$ tuned-adm active
Current active profile: aws
[ec2-user@ip-172-31-5-119 ~]$
```

Verification

The following activities were successfully completed:

- Apache was installed and activated on the RHEL EC2 instance.
- A test webpage was created and hosted using Apache.
- The webpage was made publicly accessible.
- An initial PageSpeed test was performed.
- Baseline PageSpeed results were recorded.
- Apache's `mod_deflate` module was verified.
- Gzip compression was configured.
- Apache configuration was successfully tested with `Syntax OK`.
- Gzip compression was verified using `Content-Encoding: gzip`.
- The webpage achieved **30.84% compression** in the Gzip testing tool.
- Tuned daemon was verified to be installed and running.
- The initial Tuned profile was `virtual-guest`.
- The AWS Tuned profile was applied successfully.
- The active Tuned profile was verified as `aws`.

Conclusion

Through this task, different **web and Linux performance tuning methods** were studied and tested practically on an AWS EC2 instance running RHEL.

PageSpeed was used to obtain the initial webpage performance measurements. Gzip compression was configured using Apache's `mod_deflate` module and successfully reduced the webpage response from **321 B to 222 B**, achieving **30.84% compression**. The Tuned daemon was also studied, and the system profile was changed from `virtual-guest` to the AWS-specific `aws` profile.

This task provided practical experience in **web performance testing, HTTP compression, Apache configuration, and Linux system performance tuning**.